

Лабораторная работа №2

Взаимодействие с внешним API

Студент	Ларкин Григорий Михайлович
Группа	J3213
Вариант	4 — Сервис для управления личными финансами и бюджетом
Проект	aNikral

1. Цель работы

Подключить моковое API к ранее разработанному сайту (ЛР1). Реализовать авторизацию и загрузку данных через fetch-запросы к локальному JSON-серверу вместо статически захардкоженных данных.

2. Используемые технологии

- **json-server 0.17** — имитация REST API на основе `db.json`
- **Fetch API** — выполнение HTTP-запросов из браузера
- **localStorage** — хранение токена авторизации
- **Bootstrap 5.3** — UI-фреймворк

3. Структура проекта

```
lab2/
├─ db.json           # база данных json-server
├─ package.json      # зависимости (json-server)
├─ index.html        # страница входа
├─ register.html     # страница регистрации
├─ dashboard.html    # личный кабинет
├─ transactions.html # транзакции с фильтрацией
├─ accounts.html     # счета (с добавлением через API)
├─ reports.html      # отчёты по расходам
├─ css/style.css
├─ js/
│  └─ api.js         # базовые функции apiGet / apiPost
│  └─ auth.js        # login / register / requireAuth / logout
│  └─ main.js        # темизация, утилиты рендеринга
└─ icons/sprite.svg
```

4. Структура базы данных

Файл `db.json` содержит три коллекции:

- **users** — пользователи: `id, name, email, password`
- **accounts** — счета: `id, userId, name, mask, balance`
- **transactions** — транзакции: `id, userId, description, category, date, amount`

5. Реализация авторизации

При входе выполняется `GET /users?email=...`, из результата находится пользователь с совпадающим паролем. В `localStorage` сохраняются `token` (base64 от `id+timestamp`) и `userId`. На каждой защищённой странице вызывается `requireAuth()`, который перенаправляет на `index.html` при отсутствии токена.

6. Работа с API на страницах

Страница	Запросы	Функциональность
Вход	<code>GET /users?email=</code>	Проверка учётных данных, сохранение токена
Регистрация	<code>GET /users?email=</code> , <code>POST /users</code>	Проверка уникальности email, создание пользователя
Кабинет	<code>GET /transactions?userId=</code> , <code>GET /accounts?userId=</code>	Расчёт баланса, доходов, расходов; последние 5 транзакций
Транзакции	<code>GET /transactions?userId=</code>	Список транзакций, клиентская фильтрация по описанию и категории
Счета	<code>GET /accounts?userId=</code> , <code>POST /accounts</code>	Список счетов, добавление нового счёта через модальное окно
Отчёты	<code>GET /transactions?userId=</code>	Сводка доходов/расходов, разбивка расходов по категориям с прогресс-барами

7. Запуск

```
cd J3213/Larkin_Grigory/labs/lab2
npm install
npm run api      # запускает json-server на http://localhost:3000
```

Открыть `index.html` в браузере. Демо-аккаунт: `user@example.com` / `password123`

8. Вывод

В ходе лабораторной работы была реализована имитация взаимодействия с REST API средствами `json-server` и `Fetch API`. Данные пользователей, счетов и транзакций хранятся в `db.json` и динамически

загружаются при открытии страниц. Реализована авторизация с хранением токена в `localStorage` и защитой маршрутов.